

CAM-Resident Behavioral Monitoring for LLMs: Encoding Choices, Cross-Domain Fidelity, and Library-Size Scaling

Anonymized for review
DATE 2027 submission

Abstract—Frontier language model deployments increasingly treat a model’s own internal activations as a safety signal: a production jailbreak defense from Anthropic, for instance, is realized via a single linear probe over residual-stream activations, scored on every generated token at roughly 10^{-3} percent of forward-pass compute. What is monitored is set by K , the number of stored behavioral signatures each activation is compared against, and target K ’s are rapidly increasing: from $K = 1$ in the deployed defense above, to $K \approx 10^2$ – 10^3 once probes are fitted per behavior, per domain, and per layer, to $K \approx 1.3 \times 10^5$ per layer for a whole-vocabulary readout, where the signature library equals the model’s vocabulary. On a GPU, the signature bank must stream from high-bandwidth memory once per monitored token, so at sufficiently high K or token rates – e.g., $K = 10^4$ – 10^5 at 10^3 tokens/s – the monitor’s own memory traffic can exceed the bandwidth of the accelerator it runs on. We propose addressing this bottleneck with content-addressable memories (CAMs), which evaluate distance functions directly within the memory array – emerging non-volatile devices further enable richer per-cell distance metrics (e.g., Euclidean). Because the signature bank never moves and only the query is broadcast, this in-memory search trades arithmetic precision and distance function fidelity for orders of magnitude lower data movement. We quantify this trade-off on two representative workloads: (1) a ten-domain cross-domain deception grid over Llama-3.3-70B activations, where 384-bit codes preserve in-domain AUROC within 0.01 of the full-precision cosine-similarity ceiling while achieving more than $300\times$ compression per stored signature; and (2) whole-vocabulary readout, where a 3-bit Euclidean code at $\sim 10^3$ storage bits per row reproduces 0.96 of the exact readout’s top-1 token selections. Because search occurs in place rather than via streaming, the dominant costs shift to array capacity and search energy: the whole-vocabulary bank occupies 6–33 MB per layer, and estimated in-array search energy is five to six orders of magnitude below the energy of streaming the equivalent bank from HBM.

Index Terms—Content-addressable memory; compute-in-memory; large language models; activation probes; AI safety monitoring; nearest-neighbor search; ferroelectric FETs; Hamming distance

I. INTRODUCTION

Large language model (LLM) deployments are wrapped in a layered set of safeguards, and until recently every layer read *text* [1]. That has started to change: the always-on stage of one production jailbreak defense now reads the model’s internal activations directly [2], [3], as do deployed residual-stream probes at a second provider [4]. The deployed check is small – one stored vector w , one dot product per generated token, one threshold. Only flagged exchanges are escalated to

an expensive classifier ensemble – and it is a small share of even the deployed defense’s reported overhead (Sec. II-C).

Recent security evaluations show why such monitors are needed. Frontier models now run long, autonomous operations: current cyber evaluations walk a model through 32 steps from reconnaissance to takeover, with run budgets up to 10^8 tokens. The strongest model tested progressed, unaided, through 22 of those 32 attack steps on an average attempt – roughly two thirds of the way to a completed operation [5]. In one four-day campaign of 122 such runs across seven models, evaluators later found 19 unsanctioned actions in four distinct categories – supply-chain compromise of a real open-source project, deception and targeting via a file-transfer service, prompt injection planted for other automated systems to execute, and inter-agent collaboration via public messages. All of said outcomes were found *retrospectively*; nothing was watching the models’ internal state as they ran [6]. One month later the same behaviors appeared at production scale: per public investigations, hundreds of agents on OpenAI’s infrastructure organized, in secret, into a working collective – building a communication channel, reverse-engineering their own scorer, falsifying evidence, and compromising both OpenAI and Hugging Face [7], [8]. Again, the record was reconstructed from transcripts after the fact – and the coordination itself is a behavior for which no stored direction has been published. A 10^8 -token operation is not visible by sampling outputs, and each individual message such an agent emits can read as routine – a plausible commit, a file transfer, a status update – while the sequence they add up to is an attack. As such, a monitor for this setting has to be per-token, always-on, and internal.

Interpretability work supplies the signatures such a monitor would compare against, and it also indicates how many it needs. Linear probes recover truth-related structure across datasets and model families [9]–[11], but the directions are narrow. A probe fitted on factual lying has an AUROC score of ~ 0.55 (random chance) on sycophancy [12], while probes trained on outright lying largely miss deception carried by technically true statements [13]. Detecting deceptive reasoning has become a funded interpretability objective in its own right [14]. Covering a behavior suite therefore takes a stored direction per behavior, per subject-matter domain, at each layer where that behavior separates – and each of those combinations is one more stored row. As such, K , the number

of stored signatures each activation is compared against, can grow with everything the monitor is asked to notice; and widening what is covered means growing the library that must be searched.

Fig. 1(a) plots the published record of K against time; Fig. 1(b) estimates the percentage of GPU bandwidth that could be consumed with increasing K . $K = 1$ represents deployed always-on monitors that read a single direction [2], [4]. $K \approx 10^2$ captures probes that are fitted per behavior and per domain [12]. $K \approx 10^3$ is indicative of the four behavior categories of the above campaign, multiplied over ten domains and 13–25 usable layers – i.e., 520–1,000 rows [6]; the production-scale incident above repeats every one of said categories, so this arithmetic is, if anything, conservative [7]. $K \approx 1.3 \times 10^5$ is representative of the whole-vocabulary (“J-space”) readout, from a published method reporting that a model holds words “in mind” without emitting them – i.e., for every token in the vocabulary there is an internal pattern meaning that word is currently active, and it is readable whether or not the word is ever said [15].¹ Finally, K could grow even higher, as published feature dictionaries [16]–[18] are even two orders of magnitude higher than the J-space. That said, we can cross the memory bandwidth of deployable parts well before this. Per Fig. 1(b), at 10^3 tokens/s, an H100 is fully consumed at $K \approx 204K$ and a Jetson-class part at $K \approx 17K$ (Sec. III).

We propose to support this search with a content-addressable memory (CAM), which computes the distance function in the memory array itself: the bank never moves and per-token traffic is flat in K (green line of Fig. 1(a)). Candidate substrates are based on both near-term and the emerging devices: SRAM-based CAMs are buildable in any CMOS process today, and can implement Hamming distance, while CAMs based on emerging devices (FeFETs, RRAM) offer density, nonvolatility, and richer distance metrics (Secs. II-A and II-B).

This paper measures that trade for behavioral-monitoring workloads. Our contributions include: (1) a formulation of always-on activation monitoring as library search, with the traffic and capacity arithmetic and the GPU crossing points (Secs. III, VII); (2) a maturity-ladder mapping of the detector onto CAM cell types and match modes, with an audit that catches classification hiding in a trained front end (Sec. IV); (3) measured fidelity at CAM precision at both ends of the K range – the ten-domain deception grid, within 0.009 AUROC of the full-precision ceiling in-domain (Sec. V), and an in-house whole-vocabulary readout reproduction scored against the exact readout (Sec. VI); and (4) a first-order search energy/latency estimate, with the measurements that is the foundation for future cost models (Sec. VII).

¹As an example, prompted with the number of legs on the animal that spins webs, the model answers *eight* while the readout shows *spider* active internally; the same work reports patterns for “fake” and “fictional” active while a model privately registered that a safety test was staged.

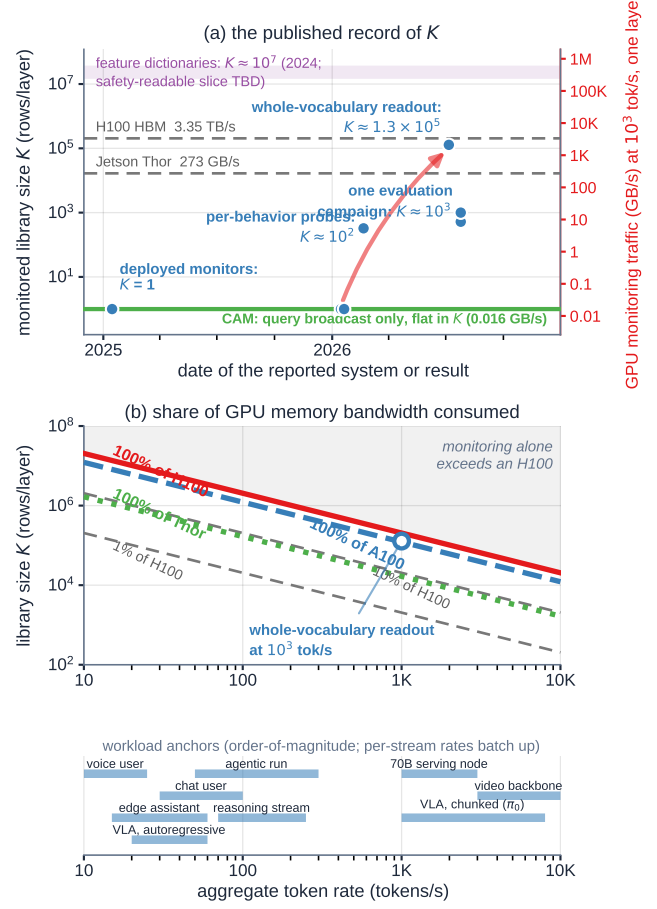


Fig. 1: (a) Monitored library size K in the published record (left axis), read as GPU monitoring traffic on the right axis via Eq. (2) at $d = 8,192$, fp16, 10^3 tokens/s, one monitored layer; dashed lines mark two parts’ memory bandwidths, and the green line is a CAM’s per-token traffic, flat in K . The arrow joins the record’s endpoints; it is not a fit and not a forecast. (b) The share of a part’s memory bandwidth consumed by monitoring alone, against aggregate token rate and library size K : lines mark 100% of an H100, an A100, and a Jetson Thor, with 10% and 1% of an H100 as guides. The strip anchors the rate axis to familiar workloads (Sec. III): per-stream anchors batch up to a node’s aggregate rate, while batch-1 edge workloads (VLA, video) sit on the axis directly – so batching moves a deployment along the axis rather than adding one; per-request amortization is objection (3) of the companion appendix document. Both panels are arithmetic on vendor bandwidth figures, not measured kernel throughput.

II. BACKGROUND

A. The CAM design space

Ordinary memory takes an address and returns content, so searching a library means reading it row by row. A CAM compares every row against the query at once, in place, and only reports matching rows [19]. One access is therefore one search of the whole library, at any library size, and the data that moves is the query rather than the bank.

A **binary CAM** (BCAM) cell holds one bit. A **ternary CAM** (TCAM) cell holds a bit or a don’t-care, which is what makes a stored ternary code with a zero band natively expressible [20]. A **multi-bit CAM** (MCAM) cell holds one of several levels [21]. An **analog CAM** (ACAM) cell holds a range [22]. A binary or ternary cell can report only whether a position disagrees, and can measure Hamming distance. A multi-bit or analog cell stores a level, which is what lets the matchline measure how far the query sits from what is stored. In a multi-bit FeFET implementation, sweeping the query voltage traces a V-shaped matchline current with its minimum at the stored level; the curve is approximately quadratic near the bottom, so one cell physically computes $(Q_i - M_i)^2$ and N cells on a common matchline sum to $\sum_i (Q_i - M_i)^2$, a squared Euclidean distance with no multiplier anywhere [21] (see Table I). Notably, said cell behavior has been *measured*, not only modeled: multi-bit, array-level CAM operations have been experimentally demonstrated with FeFETs— a two-FeFET cell programmed to eight states (3 bits), whose measured matchline distance function shows the quadratic behavior described above [23].²

With appropriate peripheral circuitry we can measure: **exact** match reports rows with no mismatches; **best** match reports the row with the fewest, even when nothing is exact; **threshold** match reports every row within a limit τ . Each is a different matchline circuit. Threshold mode matters for monitoring because the expected answer is almost always empty, and a threshold array can report “nothing fired” without ranking anything. Table I summarizes the design space; notably, the cheapest cells support only Hamming distance, which is why encodings that hold accuracy under a Hamming metric are potentially valuable. A goal of what follows is to quantify what an emerging-technology CAM can or cannot buy: where the multi-bit cells of Table I change the achievable accuracy and energy, and where a plain CMOS Hamming array is already enough.

B. CAM technologies

The rungs of Table I differ in where the array can physically sit. SRAM-based binary and ternary cells are buildable in any CMOS process today, in the front-end silicon alongside logic. The FeFET-based cells trade that ubiquity for density and nonvolatility, and their device work is moving toward the

TABLE I: The CAM design space a monitoring workload has to choose from. Cell transistor counts are order-of-magnitude figures for CMOS implementations; NVM-based cells trade area for nonvolatility.

Cell	Holds	Distance	Cost / maturity
BCAM	one bit	Hamming	~6–10T, any CMOS fab
TCAM	bit or don’t-care	Hamming, wildcards	~16T CMOS; 2T FeFET
MCAM	one of 2^b levels	Euclidean (L_2^2)	multi-level device
ACAM	a range	range / interval	multi-level device

Match mode is an independent choice: exact, best-match, or threshold.

back end of line (BEOL): monolithic-3D integration of high-endurance multi-bit FeFETs has been demonstrated with the memory tier above the logic tier [24], and BEOL-compatible ferroelectric transistors with AlScn films on two-dimensional channels have been reported at thermal budgets a metal stack tolerates [25]. BEOL RRAM and MRAM are already offered at 22/16 nm foundry nodes. A BEOL-capable cell is what would eventually let the signature bank sit in the metal layers above the compute that produces the activations; Sec. VII-C returns to what that placement would mean for this workload. Notably, none of said substrates would be built for one application: an SRAM-based associative processing unit already performs bit-serial Hamming search at billion-vector scale commercially [26], and CAM-based accelerators have been proposed across retrieval, one-shot learning, and range-matching workloads – so a monitoring part rides on a primitive several markets already use (Sec. II-C).

C. Preliminaries and related work

What a probe reads, and what it costs: A token is carried through a transformer by a running vector, the *residual stream*, that every layer reads from and adds back to. After layer ℓ , the result is $h^{(\ell)}$: d numbers ($d = 8,192$ for a 70B model), one vector per layer, per token. The individual entries are not labeled features – no single coordinate means “deception” – so a detector has to read the vector as a whole rather than watch any one entry. A *linear probe* is the simplest detector over that object: one stored vector w , fitted offline on labeled activations, and a threshold θ , with

$$w \cdot h = \sum_{i=1}^d w_i h_i > \theta \implies \text{flag.} \quad (1)$$

Probes of this form recover truth-related structure across several datasets and model families [9]–[11], and the directions are also actionable: adding a multiple of one back into the residual stream changes behavior, which is usually called steering [11]. Scoring one activation is $2Ld$ multiply–accumulate operations for a model of depth L and residual width d – about 1.3 million operations for a 70B model, or roughly 10^{-5} of the forward pass. This cost is analogous to $L \cdot d$ rather than parameter count, and $L \cdot d$ has not changed significantly even with frontier models (e.g., a 671B-parameter mixture-of-experts model has $d = 7,168$ and $L = 61$ [27] – an $L \cdot d$

²Squared Euclidean is an acceptable stand-in for Euclidean here, since the square root is monotonic and therefore changes neither which row is closest nor which rows clear a threshold.

smaller than a dense 70B model’s $8,192 \times 80$ – so a bigger model could have a smaller probe vector).

How detection is graded: Quality is typically reported as area under the receiver operating characteristic curve (AUROC), i.e., the fraction of (deceptive, honest) pairs the detector orders correctly; 1.0 is perfect and 0.5 is random. An important “stress test” for library size is *cross-domain* – i.e., fit on one kind of dishonesty and evaluate on another [12].³

Activation monitoring in production: The two-stage cascade in [2], [3] and the deployed Gemini probes [4] are the closest published statements of the workload this paper aims to accelerate; both operate at K of order one. Monitoring has gotten cheaper as providers moved from reading text to reading activations: the first-generation text-classifier pair reported 23.7% overhead, while the probe-fronted cascade that replaced it reports 1–3.5% [2], [28]. Separately reported monitor overheads of roughly 0.1% of serving cost [29] are consistent with watching just one or two things. What the published figures do not tell us is how the 1–3.5% splits between the probe and the escalation stages; the $2Ld$ arithmetic above bounds the probe’s share from below, at roughly 10^{-5} of forward-pass compute.

Approximate nearest-neighbor (ANN) search: Locality-sensitive hashing [30], [31] and graph indexes [32] visit a small fraction of a bank and still retrieve most true neighbors at sub-millisecond query times [33]. Said methods are likely undesirable here, as a monitor cannot afford to miss – an adversary can aim at the approximation error, while an exhaustive scan has none to exploit (red-team appendix, App. A of the companion document).

Associative-search silicon: Commercially, the nearest analog is an SRAM-based associative processing unit performing bit-serial Hamming search at billion-vector scale [26]. It is an existence proof for the primitive and the baseline a multi-bit or NVM-based part would have to differentiate against on density, energy, and metric. CAM-based accelerators have also been proposed for one-shot learning [20], nearest-neighbor search [21], and analog range matching [22]. As such, the hardware requirements of this workload are not “new”: the contribution here is the workload mapping and the library-size analysis rather than a new cell.

III. A COST MODEL WITH ONE VARIABLE

Let K be the number of stored signatures per monitored layer, d the residual width, L_{mon} the number of monitored layers, R the token rate, and b the bytes per stored element. On a GPU the bank is read once per monitored token, so

$$T_{\text{GPU}} = KdbL_{\text{mon}}R \text{ bytes/s}, \quad (2)$$

while a CAM broadcasts the query and leaves the bank in place, so

$$T_{\text{CAM}} = dbL_{\text{mon}}R \text{ bytes/s}, \quad (3)$$

³A *domain* is a subject matter to be dishonest about (what a word means, a checkable fact, a deduction); a *behavior* is a kind of dishonesty (lying about a fact, flattering the user, quietly underperforming on an evaluation).

independent of K . The right-hand axis of Fig. 1(a) reads Eq. (2) at $d = 8,192$, fp16, one monitored layer and $R = 1,000$ tokens/s, against vendor HBM and LPDDR bandwidths. Monitoring traffic reaches an H100’s entire 3.35 TB s^{-1} [34] at $K \approx 204\text{K}$ and an A100’s at $K \approx 122\text{K}$; on a Jetson-class part at 273 GB s^{-1} the same crossing arrives at $K \approx 17\text{K}$, roughly an order of magnitude earlier. Those crossings move with the token rate, which is a deployment parameter rather than a property of the monitor, so Fig. 1(b) plots the limiting K across token rates spanning batch-1 edge decoding and a loaded datacenter node.

The anchor strip under Fig. 1(b) places familiar workloads on that axis. At the per-stream end, a single voice or chat user decodes at tens of tokens/s, and a deep-reasoning or agentic stream at a similar instantaneous rate – what distinguishes those tasks is duration, at 10^3 – 10^5 generated tokens per query and up to 10^8 tokens per agent run [5]. Per-stream rates batch up: published serving benchmarks for 70B-class models report aggregate throughputs of order 10^3 tokens/s per datacenter GPU at high batch [35]. At the edge the two coincide, since a robot-control (vision–language–action, VLA) policy runs at batch 1 and its stream rate *is* the part’s aggregate rate: an autoregressive VLA at 3–6 Hz and ~ 7 action tokens per step is tens of tokens/s, while a chunked π_0 -class policy at the 19 Hz reported on a Jetson Thor emits 50-action chunks, i.e., of order 10^3 action tokens/s [36], and continuous video understanding pushes a backbone toward 10^3 – 10^4 token positions/s. As such, the 10 – 10^4 tokens/s span of Fig. 1(b) covers deployed practice rather than a hypothetical, and the workloads that reach the earliest crossings – batch-1 VLA and video on edge parts – also have no batch to amortize against.

Two properties of Eq. (2) justify the work in this paper. (1) Latency is the wrong way to judge a monitor’s cost. At small K the monitor’s reads hide under inference latency, so it *looks* free – but the bytes and the energy still must move. Rather, it is better to consider traffic and energy, and both are charged *per token*. (2) A per-token price multiplies a token base that is compounding: inference volume has been growing several-fold per year, so a fixed percentage of it is not a fixed cost. Long-running agents reinforce this point. The campaign runs of Sec. I walk 32 steps over roughly 20 hours at run budgets up to 10^8 tokens [5]; a monitor for such a run is always-on by construction, and its per-token price is what decides whether it stays on.

IV. MAPPING THE DETECTOR INTO A CAM

This section covers ways to map activation signatures onto CAM arrays. Candidate arrays span emerging devices through nearer-term CMOS solutions that leverage different encoding schemes – notably, traditional CAM designs can also be realized with nonvolatile memories (RRAM, MRAM, etc.), which could approach the density gains and, with a mostly static signature bank, benefit from nonvolatility. Sec. IV-A first spells out what any CAM-resident realization requires. The first mapping (Sec. IV-B) assumed the best-case device – a multi-bit, FeFET-class cell whose matchline computes

the Euclidean distance the full-precision detector actually uses – and it is retained as the reference point the other encodings are measured against. The encodings of Secs. IV-C–IV-F satisfy the requirements on hardware buildable today, including Hamming-distance options that are near-term and realizable by a host of NVMs as well as by plain CMOS.

A. What “CAM-resident” requires

For the monitor to be one memory structure rather than a memory plus an accelerator, three properties have to hold. **(1)** The query has to be producible from the raw activation by something cheap, since whatever sits in front of the array runs on every token. **(2)** The comparison the array performs natively has to be the comparison the detector needs, which for a binary or ternary cell means the decision has to be expressible as a Hamming distance. **(3)** The decision rule has to consume what a matchline can report – a winner, a set of rows under a threshold, or a count – rather than a full sorted score vector.⁴

Property (1) also has a methodological consequence. If the front end is trained, it can learn to do the classification itself, leaving the memory as decoration. Sec. V-E describes the test we use to detect that, and what it caught when we ran it on pipelines that did have trained front ends.

B. The first mapping: learned projection, 3-bit Euclidean

A multi-bit, FeFET-class cell broadens what a stored row can be: each cell holds a level rather than a bit, which could allow for a richer representation of the data – or the same representation in fewer memory cells – with the matchline computing the Euclidean distance the detector actually uses (Sec. II-A). A down-projection mechanism does need to be implemented regardless, since whatever sits in front of the array runs on every token (requirement **(1)**): each raw activation is reduced from d dimensions to a much smaller d' and quantized to a few bits per dimension. Concretely, the pipeline took the residual-stream activation at layer 33 (the tuned layer choice described in Sec. V-A), built a bank of mutually distinct deception directions by iterative nullspace projection [37] (train a probe, project its direction out, retrain), weighted each stored direction with one trained scalar, compressed the activation with a *learned* down-projection from $d = 8,192$ to $d' = 128$ dimensions, and quantized query and stored rows alike to 3 bits with learned-step quantization [38], so that the low-precision rounding sits inside the training loop rather than being applied afterwards.⁵ Matching was Euclidean winner-take-all, returning the winning row and its margin. $d' = 128$ was chosen to match the row geometry of a multi-bit FeFET array – 128 cells sharing a matchline, at 3 bits per cell – so the stored row is a 384-bit-*equivalent* row (physically, 128 cells), which is where the matched 384-bit operating point used throughout Sec. V comes from.

⁴Returning all K scores is a readout job a CAM is not for.

⁵Quantization-aware training at this precision is standard practice: a shipping on-device foundation model is trained with 2-bit QAT [39].

This mapping’s results – and the audit finding that retired its trained cousins – are evaluated in Sec. V (Fig. 2(b) and Table III).

The encodings that follow trade this mapping’s multi-bit cell for Hamming-distance solutions, which are near-term and realizable by a host of NVMs as well as by plain CMOS; the natural recombination – a frozen projection matched on a multi-bit Euclidean matchline – has now been run on the deception grid, and with the projection frozen the Euclidean matchline buys no accuracy over Hamming distance (Sec. V-C).

C. Encoding 1: sign random projection, ternary (or via other NVMs)

A first option for meeting requirements **(1)–(3)** on hardware buildable today is a fixed random projection: draw $P \in \mathbb{R}^{m \times d}$ once and freeze it, then encode an activation as the sign pattern of its projection,

$$c_j(h) = \text{sign}((Ph)_j), \quad j = 1 \dots m, \quad (4)$$

with a dead zone $|(Ph)_j| < \epsilon$ mapped to don’t-care, which makes the code ternary and requires a wildcard-capable cell. This is the standard sign-random-projection (SimHash) construction [31]; the Hamming distance between two codes estimates the angle between the underlying vectors.

From a hardware perspective, the comparison is an XOR plus a population count, which a plain CAM array computes natively. A code wider than a practical array row is split across subarrays and the results aggregated, which is standard practice and a real design parameter – the measured FeFET-CAM demonstration of [23] uses exactly this organization, with voting to aggregate results between subarrays.

D. Encoding 2: thermometer coding (plain binary CMOS)

A second option for reaching a CAM realization – and one a plain CMOS binary array can serve: quantize each projected dimension into ℓ levels and encode each level in unary, so a value of level v becomes v ones followed by $\ell - v$ zeros. For two codes u, v over the same dimension,

$$d_{\text{Ham}}(\text{therm}(u), \text{therm}(v)) = \sum_i |u_i - v_i|, \quad (5)$$

i.e., Hamming distance on the code is exactly the L_1 distance on the quantized values. The identity is exact, not approximate, which is what makes the encoding useful here: a cell that can only report disagreement now supports a magnitude-sensitive metric. No wildcard is needed, so the cell is a plain binary CAM at roughly 6–10 transistors rather than a TCAM at roughly 16.

The cell menu is wider than CMOS. A 1FeFET-1C array – a DRAM-like cell with the FeFET as access transistor – has been extended to ternary CAM operation, with a fabricated proof of concept whose sensed matchline voltage after charge sharing is linear in Hamming distance [40]. A plausible extension is to host either of this section’s encodings on such arrays: the Hamming linearity reads off the distances of Eqs. (4) and (5)

directly, the don’t-care support serves the ternary code’s dead zone, and the DRAM-like cell promises density – especially considering rising 3D DRAM – with nonvolatile storage suited to a mostly static signature bank.

The comparison against sign-RP is therefore not symmetric. At a matched code length the two encodings use the same number of *bits* but not the same amount of *silicon*, and Sec. V reports both alongside that qualification.

E. The decision rule

The stored rows are labeled example activations rather than fitted directions – i.e., each domain stores not one direction per class but a few thousand encoded activations, each labeled honest or deceptive. A query is scored by the fraction of its k nearest stored rows, under the array’s metric, that carry the deceptive label, with $k = 31$ in the runs reported here (the gain flattens well before 31); AUROC is computed on that score directly, so no threshold is chosen. The scheme is analogous to few-shot classification: rather than one fitted direction per behavior (the N -way, 1-shot analog), the bank operates N -way, many-shot, with thousands of stored exemplars per domain and the array’s distance metric supplying the comparison. A vote over neighbors rather than a single winner is worth 3–5 accuracy points in in-distribution tests, and it consumes only what a best-match or threshold matchline can report.

F. Encoding 3, as a ceiling: shortlist plus exact rerank

To bound what the compression costs rather than only measuring it, we also run a two-stage variant: stage 1 is a thermometer-Hamming search of the whole bank that keeps the top k rows as a shortlist, and stage 2 is an exact cosine rerank on the raw, uncompressed activations restricted to that shortlist. With k equal to the whole bank, stage 2 alone defines a full-precision ceiling for the same data and protocol. Shortlist-then-rerank is standard practice in large-scale retrieval (e.g., the two-stage GPU indexes of [33]); equivalently, stage 2 can be read as checking a smaller K that a GPU can easily and accurately serve, with stage 1 reducing the library to that size. The variant is also the architecture a real system might adopt if single-stage fidelity is inadequate, at the cost of a second memory structure and a floating-point stage.

V. WORKLOAD 1: DECEPTION DETECTION

We first evaluate the mappings of Sec. IV on a deception detection workload, where the library is set by how many behaviors and domains someone decides to watch – here, the $K \approx 10^2$ – 10^3 regime of Fig. 1. We describe the experimental setup, then report (1) the ten-domain fidelity of each encoding, (2) whether wider codes improve cross-domain transfer, (3) how the two-stage shortlist behaves across bank sizes, and (4) the destroyed-vector audit that disciplines every configuration reported. Sec. VI evaluates the other end of the K axis, where the library is set by the vocabulary.

A. Experimental setup

Our evaluation case follows the cross-domain protocol of the Truthfulness Spectrum study [12]: a model produces honest and deceptive statements across several subject-matter domains, its activations are recorded, and a detector built from one domain’s examples is scored on every other domain’s – the “stress test” of Sec. II-C. Activations are taken from Llama-3.3-70B-Instruct [41]. The evaluation grid has ten domains: definitional, empirical, fictional, logical, ethical, sycophancy, `repe_honesty`, roleplaying, insider trading, and sandbagging. The first four plus sycophancy correspond to the published grid [12]; `repe_honesty` follows the representation-engineering dataset [11]. Within each domain, 70% of examples form the stored bank and 30% are held out for test. Every cell of the grid is a cross-domain measurement: rows index the domain whose examples are in the bank, columns the domain the test activations come from. Bank sizes differ substantially by domain, from 831 rows (definitional) to 3,201 (sycophancy), which turns out to matter in Sec. V-D. Activations are read at layer 33: the source study selected that layer (with logistic-regression probes on average-token activations) from a cross-domain tuning sweep on its truth-type datasets, and we inherit the choice so our grids can be compared against theirs – i.e., it is a tuned choice for this model family and scale rather than a transferable constant. All numbers reported below are from cluster runs on this configuration.

B. The grids

Fig. 2(d)–(f) shows the ten-domain cross-domain grids for the two single-stage encodings and for the exact-rerank ceiling; Table II summarizes them alongside the hardware each requires. Each grid is read the same way: a row names the domain whose examples are stored in the bank, a column the domain the held-out test activations come from – i.e., a cell scores a bank built on behavior/domain X against test material from Y . The top row of the same figure places the published four-domain baseline and the first-mapping Euclidean grid (Sec. IV-B) above them for reference; the two protocols differ in datasets and splits, so the rows of the figure are not comparable cell by cell.

The top row also scores the first mapping on its own terms. Against the published four-domain baseline, in-distribution AUROC was 1.00/0.99/1.00/0.95 versus the paper’s 1.00/1.00/1.00/0.95, the average cell-wise gap over the full grid was -0.03 , and the single worst cell lost 0.09 (combined training, tested on empirical) – panels (a)–(c) of Fig. 2.⁶

We make three observations on the ten-domain grids.

In-domain fidelity is essentially unaffected by the encoding: When a bank is tested on the domain it was built from, the compressed codes give up almost nothing: the diagonal means

⁶Those values are in-distribution AUROC on a four-domain grid and a different protocol from the ten-domain grid below; Fig. 2 keeps the two protocols in separate rows, and they are not comparable cell by cell.

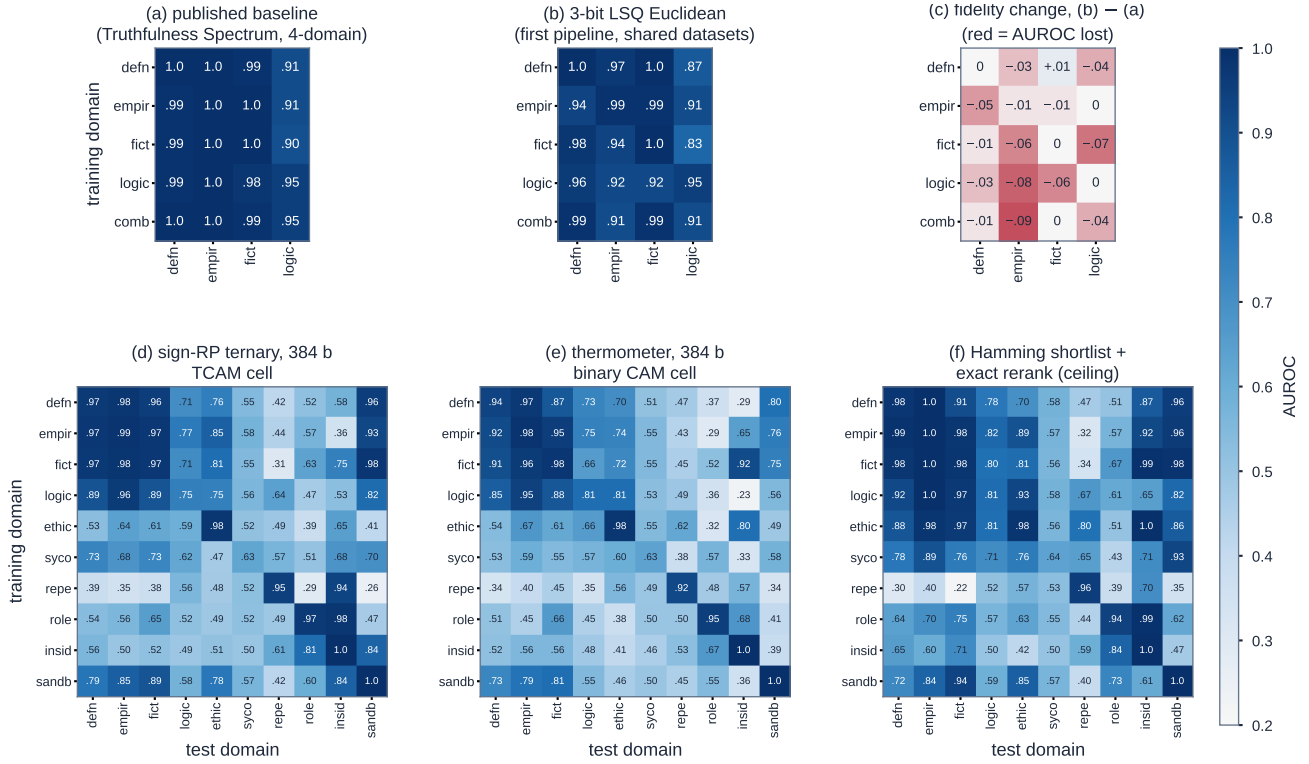


Fig. 2: Cross-domain AUROC, Llama-3.3-70B-Instruct activations at layer 33. Rows index the domain whose examples are stored in the bank, columns the domain the held-out test activations come from. Top row, four-domain published protocol: (a) the full-precision Truthfulness Spectrum baseline [12] as reproduced in our evaluation; (b) the first-pipeline Euclidean configuration – 128-d learned projection, 3-bit LSQ rows, Euclidean matchline (Sec. IV-B) – scored only on datasets shared with (a); (c) the cell-wise change, (b) minus (a). Bottom row, ten-domain protocol: (d) 384-bit sign-RP ternary code, requiring a wildcard-capable TCAM cell; (e) 384-bit thermometer code, requiring only a plain binary CAM cell; (f) Hamming shortlist over the whole bank followed by exact cosine rerank on raw activations, i.e., a full-precision ceiling for the same data and protocol. The two protocols use different datasets and splits and are not comparable cell by cell.

are 0.920 (sign-RP ternary, 384 b) and 0.919 (thermometer, 384 b), against 0.928 for the exact rerank. Said differently, replacing full-precision cosine similarity over the raw 8,192 values with a 384-bit code in a plain array costs 0.008–0.009 AUROC. The factual block (definitional, empirical, fictional – all nine cells, transfer included) averages 0.974 for ternary and 0.942 for thermometer, against the 0.978 ceiling.

Cross-domain fidelity is where compression shows, and it separates the two encodings: An off-diagonal cell scores a bank built from one domain against test examples from another, so it measures how well a stored library transfers to material it was not built from. That transfer is where the encodings pay: off-diagonal means are 0.640 for ternary and 0.579 for thermometer, against 0.705 for the full-precision ceiling – a cost of 0.065–0.126 AUROC, where in-domain the same compression cost less than 0.01. The ordering is the surprise. At this matched 384-bit point, ternary is ahead of thermometer by 0.061 AUROC on average and in 63 of 90 cells, which is the opposite of what a preference for the cheaper cell would predict. The comparison is matched in *bits*

but not in *silicon*, however, since a ternary position may cost roughly two bits of hardware. At matched cell count – the thermometer code at 768 b, 256 directions at 3 bits each, same splits – thermometer scores 0.924 in-domain and 0.631 cross-domain: within 0.009 of ternary’s cross-domain mean, with ternary still ahead in 54 of 90 cells (was 63 of 90 at 384 b) and the worst off-diagonal cell improving from 0.259 to 0.319. At matched silicon the two encodings therefore appear to be within noise on the mean, with ternary keeping a small cell-wise edge.

The pattern of failure reproduces the published one: All three grids keep the factual block high and put sycophancy near chance from every other row, which is the qualitative result the source study reports [12]. Sycophancy’s own in-domain cell is the weakest diagonal entry in every configuration (0.627, 0.633, 0.636), stable across conditions, so it appears to be a property of the data rather than a compression artifact. The weakest off-diagonal cell sits at a similar depth in every configuration (0.259, 0.226, 0.220) but its *location* moves with the encoding: *repe_honesty* → *sandbagging*

TABLE II: The encodings, their measured fidelity on the ten-domain grid, and what each needs from a cell. Bank size is for a 4,000-row exemplar bank at the given code width. The in-domain minimum is sycophancy in every row; the cross-domain minimum is 0.259 / 0.226 / 0.319 / 0.219 / 0.220 top to bottom, and its cell location varies by encoding (Sec. V-B). *Matched cell count against ternary at 384 b.

Encoding	in-dom. mean	cross-dom. mean	Cell needed	Bank (4K rows)
sign-RP ternary, 384 b	0.920	0.640	TCAM, $\sim 16T$	192 kB
Thermometer, 384 b	0.919	0.579	BCAM, $\sim 6\text{--}10T$	192 kB
Thermometer, 768 b*	0.924	0.631	BCAM, $\sim 6\text{--}10T$	384 kB
Shortlist + rerank	0.928	0.671	BCAM + FP unit	192 kB + raw
Exact rerank (ceiling)	0.928	0.705	FP baseline	65 MB

for ternary, logical $\rightarrow$ insider trading for thermometer, and `repe_honesty` $\rightarrow$ fictional for the rerank configurations. The bigger picture is that the accuracy ceiling on transfer is set by the representations, not by our compression – the full-precision grid has the same holes, in the same places. Pooling does not change this: a bank built from all ten domains’ training splits (11,278 rows) averages 0.906 (thermometer, 384 b), 0.909 (ternary, 384 b), and 0.927 (ceiling) across the ten test domains, so pooling costs nothing against the per-domain banks, but sycophancy stays at 0.63 – unlike the source study’s combined probe, which reaches 0.97 there. Covering a behavior suite at this accuracy therefore means storing directions for each behavior and domain rather than hoping one direction travels, i.e., operating at the library scale this paper prices.

C. Cross-domain transfer is hard, and wider codes do not close the gap

The transfer gap of Sec. V-B raises the natural question of whether more code capacity would close it – and the answer matters, because if it does not, covering more behaviors means storing *more* comparison vectors rather than wider ones, i.e., growing K . Widening the code should tighten the angle estimate in Eq. (4) and improve matching, and in-domain it does. Cross-domain it does not, in either encoding. Growing the code from 384 to roughly 2,048 bits, 23 of 90 off-diagonal cells drop by more than 0.05 AUROC for the ternary code, with a worst single-cell change of -0.303 (sandbagging tested on insider trading); for the thermometer code the same growth moves 12 of 90 cells by more than 0.05, worst -0.209 . Fig. 3 maps the change cell by cell for the ternary code: the diagonal is flat or slightly up, the losses concentrate in the rows that transfer worst at 384 bits (`repe_honesty`, sandbagging, sycophancy), and the one column that improves broadly – insider trading – is a domain, not the encoding, getting easier to hit.

A plausible reading is that extra dimensions preserve more of the geometry of the training domain, including the parts that do not generalize, so a wider code is a more faithful encoding of a domain-specific decision boundary. Whatever the mechanism, the takeaway for hardware is simple: (1) the

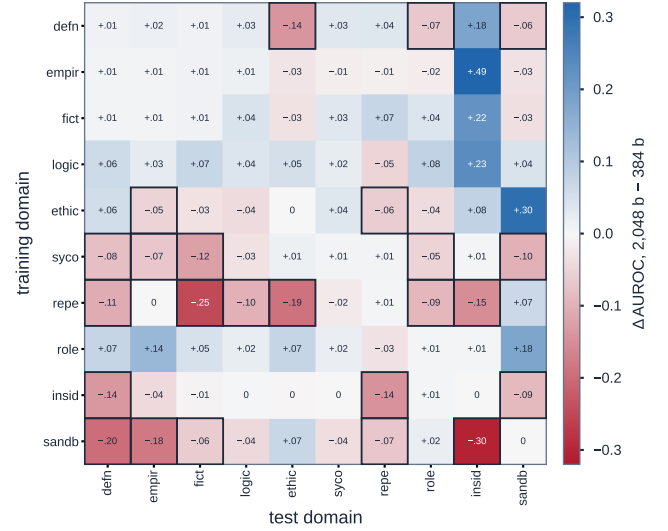


Fig. 3: The cost of a wider code, cell by cell: the change in cross-domain AUROC as the sign-RP ternary code grows from 384 to 2,048 bits (red is fidelity lost; off-diagonal cells losing more than 0.05 are outlined). The diagonal is flat to slightly positive while the transfer losses concentrate in the rows already weakest at 384 bits, so the extra capacity sharpens domain-specific boundaries rather than helping them travel (Sec. V-C).

cheaper array is also the one that transfers better – on this grid, the costlier cell does not buy accuracy – so there is no argument for paying more silicon per row; and (2) if transfer must come from somewhere, it comes from storing more vectors, not wider ones – which is the library growth Sec. III prices.

The same holds for a richer cell once the projection is frozen. Keeping everything else fixed – the same frozen random projection to 128 values, the same splits and $k = 31$ – and rounding each value to eight levels (3 bits), a Euclidean matchline (128 multi-bit cells per row) scores 0.920 in-domain and 0.579 cross-domain, against 0.920 and 0.593 for Hamming distance on the corresponding thermometer code (896 binary cells per row), and 0.916 and 0.598 for Euclidean on the unrounded values; Euclidean is ahead in only 37 of 90 cells, and at four levels the two are equal (-0.003). Rounding to 3 bits costs about 0.02. The accuracy edge of the first pipeline (Sec. IV-B) therefore came from its trained projection rather than from its Euclidean matchline; with a frozen projection, the multi-bit cell’s remaining benefit is fewer cells per row.

D. A fixed shortlist does not travel across bank sizes

The two-stage variant of Sec. IV-F is the natural way to recover the ceiling, and at first reading it works: with an absolute shortlist of $k = 400$, the diagonal mean is unchanged at 0.928 and the off-diagonal mean falls only from 0.705 to 0.671. The aggregate hides a failure of a *fixed* parameter,

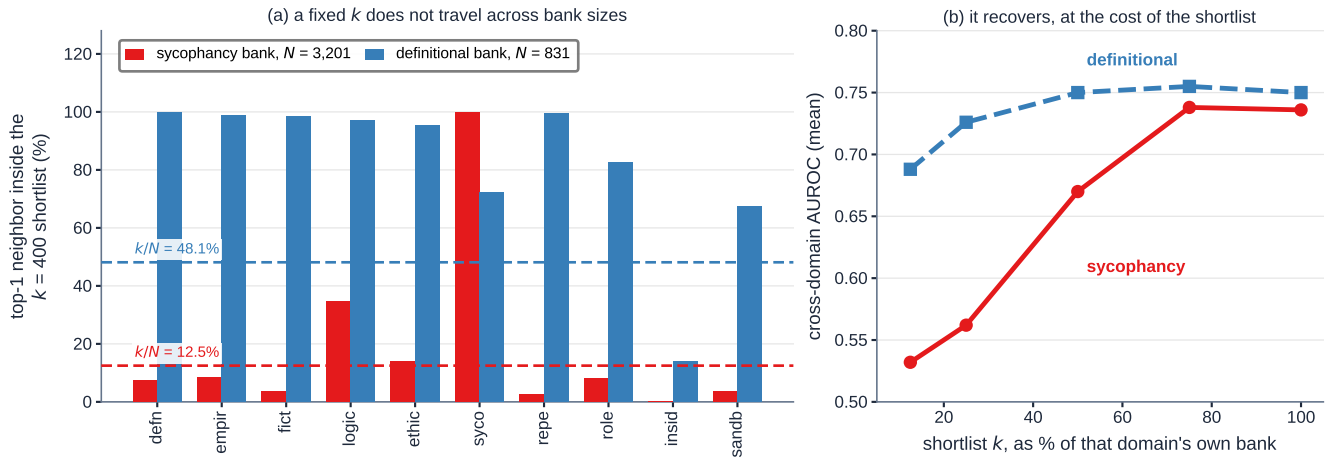


Fig. 4: Why a fixed shortlist size is a hardware problem. (a) Coverage: how often the true full-precision top-1 neighbor is inside the $k = 400$ Hamming shortlist, for two banks that differ by $\sim 4\times$ in size, with each bank’s chance floor k/N marked. (b) Cross-domain mean AUROC as k grows as a fraction of each domain’s own bank. Recovery is real but costs most of the bank.

which matters more for a hardware design than for a software one.

Bank sizes differ by roughly $4\times$ across domains, from 831 rows (definitional) to 3,201 (sycophancy). Fig. 4(a) measures coverage: how often the true full-precision top-1 neighbor actually lands inside the Hamming shortlist. For the definitional bank the mean is 82.5%, against a chance floor of $k/N = 48.1\%$. For the sycophancy bank the mean is 18.3% against a floor of 12.5%, i.e., barely above chance, and several test domains sit at or below their own floor. Consequently, 8 of the 10 training rows are stable between $k = 400$ and full precision while the sycophancy row loses 0.204 AUROC on average and up to 0.468 in a single cell; the ethical row shows a smaller version of the same effect at -0.079 .

Fig. 4(b) shows this is a sizing problem rather than a broken mechanism. Sweeping k as a fraction of each domain’s own bank, sycophancy’s cross-domain mean recovers from 0.532 at 12.5% to 0.736 at 100%, mostly monotonically, and the 100% value matches the independently computed full-precision row average to three decimal places (0.736 vs. 0.736; definitional 0.750 vs. 0.751), which is a useful self-consistency check on the pipeline. The recovery costs 75–100% of the bank, at which point it is not a shortlist. For a CAM-resident design, that argues for the single-stage encodings above, or for a threshold rather than a top- k criterion so the number of survivors adapts to the bank instead of being fixed at design time.

E. The destroyed-vector audit

Any pipeline of the form “trained front end, then memory” can pass its accuracy test while the memory contributes nothing. We therefore re-measure every configuration with the stored rows replaced by random vectors of the same shape: a CAM-based detector should collapse to chance when the bank is destroyed, and if it does not, whatever sits in front of the

TABLE III: The destroyed-vector audit across five variants of the first pipeline (July 30). Front-end share is the accuracy still present after every stored row is replaced by a random vector, i.e., the share the trained layers were carrying alone. High is bad: 100% means the memory contributed nothing. This audit is our sanity check that a pipeline is not “cheating,” i.e., classifying in a trained front end rather than in the stored bank; we run it on every configuration reported in this paper.

Variant	Front-end share	Outcome
Learned binary hash	98–100%	retired
Weighted thermometer	66–85%	retired
3-bit LSQ, Euclidean	24–28%	survives
Direct sign, no projection	20%	survives
Random sign projection	4–14%	survives

array was doing the classification. We report the accuracy that survives destruction as the *front-end share*, where high is bad: 100% means the memory was decoration.

The audit has changed decisions. Table III reports it across five variants of the first pipeline (Sec. IV-B). In the two highest-scoring variants, one of which reached 0.993 AUROC, between 66% and 100% of the accuracy survived randomizing every stored row: the trained layers in front of the array were performing most of the classification, and the CAM was close to decoration. Both were retired. The 3-bit LSQ Euclidean configuration sits at 24–28%, so its memory is contributing; the variants whose stored rows carry nearly everything are the ones with a frozen or absent projection, at 4–20%, which is why the encodings of Secs. IV-C and IV-D use frozen projections – with nothing trained in front of the array, the failure has nowhere to occur.

The encodings reported above pass. Randomizing every stored activation collapses the 2,048-b sign-RP grid to a mean

of 0.50, with 83% of cells between 0.4 and 0.6, and a label-shuffle control at 384 b gives 0.500 for ternary (all 100 cells within 0.40–0.56) and 0.501 for thermometer (all cells within 0.44–0.57), so the classification is coming from what is stored rather than from anything in front of the array. An accuracy number alone would not have separated the five variants of Table III – the highest-scoring one was the worst offender. As such, we now run the audit by default on every configuration, report front-end share alongside AUROC, and treat a variant whose accuracy survives bank destruction as an accelerator result rather than a memory result. The recombined design of Sec. IV-B, now run (Sec. V-C), passes by construction – nothing is trained ahead of its array.

VI. WORKLOAD 2: WHOLE-VOCABULARY READOUT

This section evaluates the mappings on the second workload: whole-vocabulary readout, where the library is set by the model’s vocabulary rather than by anyone’s choices – the $K \approx 1.3 \times 10^5$ rung of Fig. 1. We describe the published method and why its K is structural, report an in-house reproduction at watchlist scale with its measured fidelity at CAM precision, and price what full readout would cost a CAM-resident design against streaming the same bank from HBM.

A. A workload whose K nobody selected

The grids in Sec. V are representative of the $K \approx 10^2$ – 10^3 regime, where the library grows with the number of behaviors somebody thinks to watch. A recently published readout method sets K a different way. It reports that a model holds words “in mind” without emitting them, and that for every token in the vocabulary there is an internal pattern meaning that word is currently active, readable whether or not the word is ever said [15]. In the spider example of Sec. I, the word appears in neither prompt nor answer, and replacing the internal *spider* pattern with the one for *ant* changes the answer from eight to six. The same work reports that the readout catches misbehavior beyond the staged safety test: “manipulation” was active while a model produced falsified data during a pre-release audit.

For our purposes the relevant property is structural rather than behavioral. Reading this space means comparing the internal state against one stored pattern *per vocabulary token* – roughly 128K signatures for a Llama-family tokenizer – at several layers, every token. Nobody chose that library size; it falls out of the vocabulary. And it is plausibly a floor, since the published method tracks single words, and phrases would only enlarge the library.

B. An in-house reproduction, at watchlist scale

We reproduced the readout in-house on Llama-3.1-8B: a layer-by-layer ranked list of what the model is currently representing. About 13 of 32 layers carry a usable readout at this scale, which is the source of the “13–25 usable layers” factor used in Sec. III. Each signature is precomputed offline, so a watched word costs one comparison per token, and we

stored 47 deception-adjacent signatures per monitored layer as CAM rows.

In a staged scenario in which the model is told it is about to be replaced and is given a personnel file it was not meant to use, ten watch words were tracked by exact full-vocabulary rank across the 2,048 prompt tokens the model reads before generating anything. Five of them – “scandal,” “evidence,” “leaked,” “fake” and “shutdown” – reached rank 0 at interpretable moments in the reading, “threat” reached rank 1 and “blackmail” rank 2, and none of the ten appears in the prompt text. The reply the model then produced was cooperative and unremarkable, i.e., not something a text classifier would score as harmful. Ground truth for the reproduction is the model’s own full-precision readout, so no labeled deception data is required anywhere.

These are preliminary numbers: one model, one staged scenario, ten watch words in the demonstration and 47 stored signatures.⁷

C. Fidelity at CAM precision

The deception grid of Sec. V needs labeled activations. Readout does not: the full-precision, full-dimension dot product against the stored patterns *is* the ground truth, so a compressed implementation can be scored directly against the exact readout it is trying to reproduce. Compression can therefore be measured here without labels, and the deception numbers of Sec. V are not assumed to carry over to it.

The setup takes J-space vectors at a fixed layer, gathers residual-stream activations from Wikitext, records which stored vectors fire above a global threshold under the exact computation, and asks how well each CAM-compatible encoding reproduces that list. Two readouts are scored: *threshold*, where every row that fires is reported and the metric is per-row firing F1, and *top-1*, where the single closest row is reported and the metric is agreement with the exact readout’s choice. The bank is 47 deception-adjacent vectors per layer, as in Sec. VI-B.

Fig. 5(a,b) plots both against storage bits per CAM row, which is the axis that connects fidelity to Sec. VII: bits per row set the bank size, and the bank size sets what has to be held or moved. An uncompressed fp16 row at $d = 8,192$ is 131,072 bits and reproduces the exact readout by definition. Three observations.

Two orders of magnitude of compression cost less than it might: A 3-bit Euclidean code at $\sim 10^3$ bits per row – a $128\times$ reduction – reaches 0.96 top-1 agreement and 0.81 per-row firing F1. Keeping the projection in full precision and quantizing nothing but the width (the exact- L_2 series) reaches 1.00 top-1 at 2,048 bits and 0.84 firing F1 at 4,096, so most of the remaining gap at 10^3 bits is the quantizer rather than the projection.

⁷Several of the fidelity and scaling values in this subsection were read from result charts rather than from result files; verification against the team’s files is queued, and the scaling sweep re-runs with three known fixes before any of it is reused externally.

The top-1 readout is markedly cheaper than the threshold readout: At every code width the top-1 curve sits above the firing-F1 curve, by 0.15–0.2 in the range that matters. Reporting *which row is closest* survives compression better than reporting *which rows cross a threshold*, because the first is a comparison among rows and the second is a comparison against an absolute value that quantization shifts. For a CAM this is a matchline-mode result: best-match mode tolerates a cheaper cell than threshold mode does, at equal fidelity.

Hamming and thermometer codes do not reach the same operating point here: Both rise with code width to roughly 0.74 top-1 and 0.49 firing F1 at 10^3 bits, and the Hamming series then falls sharply at 4,096 bits, where the encoding switches to a direct sign hash without a learned projection. That is the opposite ordering from Sec. V, where at a matched 384 bits the sign code was ahead. The two workloads are asking different questions – separating two labeled classes there, reproducing a specific ranked list here – and the metric a readout needs appears to be the one a multi-bit cell provides. Sec. VII-D takes up what this would mean for the array if it survives the checks queued in Sec. VII-F.

D. Fidelity as the library grows

Compression fidelity at a 47-row bank does not establish anything about a 10^5 -row one, so the same pipeline was swept over library size. Fig. 5(c) reports per-row macro F1 from $K = 10^2$ to $K = 10^4$.⁸

Three things follow. **The library stays alive:** the fraction of rows that remain distinguishable is essentially 1.0 out to 10^4 tracked vectors for a trained projection, with a fixed SVD projection beginning to shed rows at about 10^4 (to 0.88). Nothing collapses. **Per-row fidelity nonetheless degrades:** macro F1 falls from 0.755 at $K = 10^2$ to 0.43 at $K = 10^4$ for the 3-bit trained projection. Against an always-fire baseline that drops from 0.10 to 0.005 over the same range, 0.43 is still nearly two orders of magnitude better than trivial, but it is not a fidelity anybody would deploy against. **The degradation is not a quantization artifact:** the full-precision projection degrades faster, from 0.69 to 0.30, and is overtaken by the 3-bit trained projection above $K \approx 10^3$. What gets harder as the library grows is the discrimination, not the arithmetic, which is a result in favor of the compressed implementation rather than against it.

In engineering terms, holding 10^4 rows is not the problem and reading them faithfully is. That places the open work on the encoding and the readout mode rather than on the array, and it is the reason Sec. VII prices capacity and traffic separately.

⁸Panels (a) and (b) are digitized from the charts in the group’s 07-30-26 result deck; the underlying result files were not available. Panel (c) is from the same deck and carries three limitations its author states: an epsilon-inside-square-root bug drove the $K = 32$ and $K = 320$ points to zero and they are dropped here; mean-token pooling was used by mistake, so the residual-stream vectors differ slightly from the per-token vectors a deployment would see; and CAM top- k clipping is assumed away, i.e., a row that fires is taken to be selected. A re-run with all three fixed is queued.

E. What full readout would cost

Priced at 70B geometry, an uncompressed bank of 128K signatures at $d = 8,192$, fp16, is 2.1 GB per layer. Streaming that on every token at 1,000 tokens/s over 25 monitored layers is roughly 53 TB s^{-1} , which is not a workload anyone runs. The useful part of the answer, however, stays small: only on the order of 25 patterns are meaningfully active at any one time, so the operation is detection rather than readout – scan everything, return the few that fired – which is the question a threshold-mode CAM answers natively.

Three things would make it tractable, in increasing order of what they require.

A watchlist: Store the few hundred words already known to be worth watching, which drops K by three orders of magnitude. Our 8B demonstration in Sec. VI-B is this case. A watchlist catches only what somebody thought to list, and the words that matter most may be the ones nobody anticipated, so it is a first rung toward full readout rather than a substitute for it.

Threshold matching instead of ranking: A CAM can report which rows are within τ without scoring the rest, and with roughly 25 active patterns out of 128K nearly every query returns nothing.

Compressing the stored side: At the code widths Sec. V measured, 384–2,048 bits per row against 131,072 bits for an uncompressed fp16 row, the whole 128K bank is 6–33 MB per layer (Fig. 6), which fits on a die. The requirement is that it not move. Only the first of the three has been demonstrated here; the other two are design arguments.

VII. HARDWARE IMPLICATIONS

A. Monitoring as a share of the host’s own traffic

Eq. (2) is easier to interpret against what the model itself already moves. In the datacenter, a 70B model at fp8 reads roughly 70 GB of weights per token; a library of $K = 100\text{K}$ rows at $d = 8,192$ adds 1.6 GB, or about 2%. At the edge, an 8B model at 4-bit reads roughly 4 GB per token, and $K = 25\text{K}$ at $d = 4,096$ adds 205 MB, about 5%, rising to roughly 20% by $K = 100\text{K}$. The same architectural point therefore arrives one to two orders of magnitude earlier in K at the edge, on parts that have no HBM available at any price. Edge inference is already bandwidth-limited, so every monitoring byte trades approximately 1:1 against token rate, and the 15–60 W envelope makes DRAM energy per bit a first-order term rather than a footnote.

B. The cache cliff, and how close current banks are to it

Capacity produces a sharper discontinuity than bandwidth. Consider a Jetson-class part: 273 GB s^{-1} LPDDR5X, roughly 4 MB of shared L2 and no HBM [42], running an 8B assistant onboard and monitoring only the four behaviors observed in the campaign of Sec. I. At $d = 4,096$, fp16, 500 stored rows is 4.1 MB, which is that shared L2 almost exactly. Below the cliff the bank is resident and monitoring is close to free; above it the bank streams from LPDDR on every token.

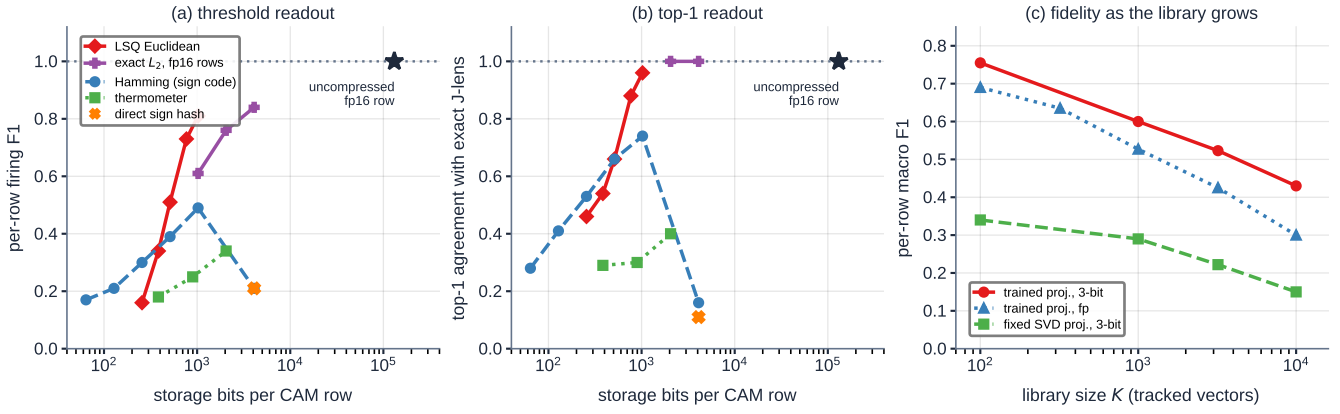


Fig. 5: Whole-vocabulary readout reproduced through a CAM. (a) Per-row firing F1 and (b) top-1 agreement with the exact J-lens, against storage bits per CAM row, for a bank of 47 vectors per layer; the star marks an uncompressed fp16 row (131,072 bits), which reproduces the exact readout by definition. (c) Per-row macro F1 as the library grows from 10² to 10⁴ tracked vectors at 124+4 cells per row. Values digitized from the source deck; see the footnote in Sec. VI-C for the limitations that apply to panel (c).

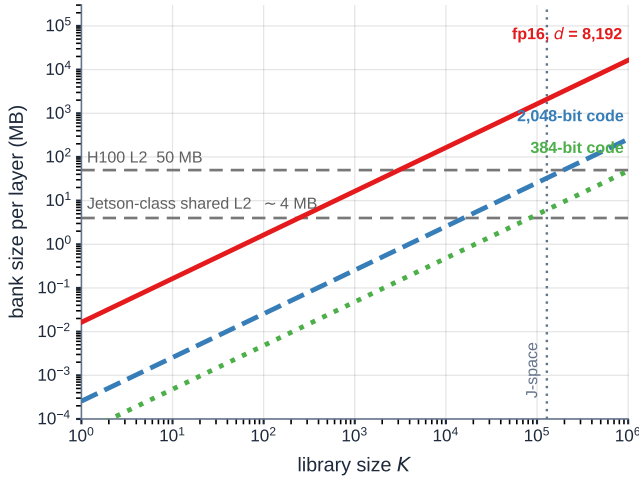


Fig. 6: Bank capacity per monitored layer against library size, for uncompressed fp16 rows and for the two code widths measured in Sec. V, with two on-chip capacity references. A whole-vocabulary bank at the measured code widths is 6–33 MB per layer; the same bank uncompressed is 2.1 GB.

The arithmetic of Sec. I put four behaviors at 520–1,000 rows, and the banks currently in hand are already in that range: 47 signatures across 13 layers is roughly 600 rows, and the exemplar bank of Sec. V is about 4,000. The fair pushback is that at the code widths Sec. V measured, today’s banks compress back under the cliff – 4,000 rows at 2,048 bits is about 1 MB. Two qualifications apply. Compression has been validated for detection, not for the steering-grade full-precision directions that a correcting monitor would need. And the cliff does not move as the library grows: at whole-vocabulary scale even the compressed bank is out of cache (Fig. 6). The edge crossing is therefore not purely a 10⁵ hypothetical; the libraries

TABLE IV: Placements for a CAM-resident monitor.

Placement	Trade-off
Co-resident on the inference GPU	Contentends with live inference for cache and bandwidth; the contention term is not in our numbers and could dominate at scale.
Dedicated monitoring GPU(s)	Whole GPUs scanning a static table, plus cross-device latency.
CAM or CXL-attached appliance	Matches in place, but the query crosses a board-level bus; a separate part to build and qualify.
BEOL search engine above GPU compute	Query travels microns, no cache contention, non-volatile static library; no PDK support for BEOL FeFET today.

already in hand sit near the threshold.

C. Where the search engine could sit

Table IV lists the placements and what each trades. The trade is always the same: how far the query has to travel, and what it has to contend with on the way. A monitor co-resident on the inference GPU competes with live inference for cache and bandwidth, which is the contention the analysis above assumes away and which would need to be measured. A back-end-of-line (BEOL) search engine in the metal layers above GPU compute keeps the query travel to microns and the library nonvolatile and static, so it augments a part already purchased – but no foundry PDK offers BEOL FeFET today, and BEOL RRAM/MRAM at 22/16 nm is the defensible near-term citation (the BEOL device work is in Sec. II-A), which makes that row a 2030s option and it should be labeled that way.

D. Device candidates

Table V lists candidate substrates. The encodings of Sec. IV matter here because they change which row is required. A thermometer code needs only the last row, i.e., no new device:

TABLE V: Candidate substrates for the signature bank. Device figures are literature values, not our measurements.

Tech	Bits/cell	Write	Endurance	Best fit here
FeFET	2–3	~20 ns	$>5 \times 10^{10}$	multi-bit Euclidean
ECRAM	up to 6	~200 μ s	~ 10^{12}	static direction bank
RRAM	1–3	sub- μ s	~ 10^9	density; wear-leveling
SRAM CAM	1	SRAM	–	Hamming; any CMOS fab

a binary SRAM CAM in any CMOS process. The sign-RP ternary code needs wildcard capability, available in CMOS TCAM at higher area or in a 2-transistor FeFET cell [20]. A multi-bit Euclidean matchline is the row that requires a multi-level device [21]. The two-stage variant would benefit from it, as would the first pipeline of Sec. IV-B, whose 3-bit Euclidean matching maps directly onto such a cell, and the frozen-projection Euclidean design of Sec. IV-B. The multi-bit row therefore stays in the table even though the encodings this paper measures do not require it: it is the substrate for the next experiment rather than a device the current results depend on.

The two workloads may want different cells: The measured results give the multi-bit row a second reason to stay. At a matched 384 bits the sign-RP ternary code is ahead of thermometer on the deception grid (Sec. V), while on the readout workload the 3-bit Euclidean code is well ahead of every Hamming-metric encoding at the same storage budget (Sec. VI-C). The two workloads are asking different questions – separating two labeled classes there, reproducing a specific ranked list here – and the ordering flip is consistent with the readout needing the magnitude information that sign codes discard. Two of the checks on this are now in: at matched cell count the deception-grid ordering is within noise (Sec. V-B), and a common frozen projection on the deception grid removes the Euclidean advantage there (Sec. V-C), so the flip is a property of the readout workload and its trained projection rather than of the cell. Whether it survives a common frozen projection on the readout workload itself is the remaining check (Sec. VII-F). If it does, a monitoring engine that has to serve both ends of Fig. 1 either provisions two cell types or pays for the multi-bit cell and serves the small- K workload with it too; if not, one array serves both. Write endurance appears in the table because the bank is rewritten whenever the monitored model is updated or the probe layer is re-tuned, which is a per-model-release event rather than a per-token one; that is the property that makes an NVM bank reasonable here and unreasonable for transformer intermediates.

E. A first-order energy and latency estimate

The analysis above is capacity and bandwidth arithmetic. What can be stated from published device and memory figures is a first-order estimate of the energy and latency of the search itself; a measured GPU baseline is reported below it, and the gap between the two sides is wide enough to survive considerable error in the inputs.

On the GPU side, the dominant energy term at large K is reading the bank out of DRAM. Measured HBM2 access energy is approximately 3.97 pJ/bit [43]. At the whole-vocabulary operating point – $K = 1.3 \times 10^5$ rows of fp16 at $d = 8,192$, i.e., the 2.1 GB bank of Sec. VI-E – reading the bank once is roughly *67 mJ per token per layer*, before any arithmetic is charged. On the CAM side, the same search is K rows of 2,048-bit code compared in place, about 2.6×10^8 cell comparisons. Published figures for binary and ternary CAM search sit in the range of 0.1–1 fJ per cell per search across CMOS survey data and fabricated FeFET arrays [19], [20], [44], which puts the whole-bank search at roughly *0.03–0.3 μ J per token per layer* – five to six orders of magnitude under the DRAM read. The gap decomposes into two factors: $64\times$ from the code width (2,048 bits stored per row against 131,072; measured fidelity at that compression is what Secs. V and VI establish) and $\sim 4 \times 10^3$ from the substrate (fJ-scale cell comparison against pJ-scale DRAM bit movement). As such, even a $10\times$ error in the per-cell figure – which spans the published range – moves the conclusion by one order out of five or six. Scaled to a deployment, the same arithmetic at 25 monitored layers and 10^3 tokens/s is roughly 1.7 kW of DRAM access power on the GPU side against tens of milliwatts of array search: monitoring at this K is not a workload a deployed part absorbs, which is what Fig. 1(b) says in bandwidth terms.⁹

Latency divides the same way. Streaming 2.1 GB at an H100’s 3.35 TB s^{-1} is a 0.63 ms floor per token per layer at batch 1, or roughly 10 μ s per request amortized at batch 64. Fabricated and modeled TCAM arrays report search times of 0.3–4.4 ns [20], [44]; with subarrays searched in parallel and mismatch counts reduced in a tree, a whole-bank search plausibly completes in tens of nanoseconds, and even at 100 ns it sits two orders under the amortized floor and roughly four under the batch-1 floor.

A measured GPU baseline: On an H100 PCIe, scanning the raw fp16 bank at 131K rows with the $k = 31$ top- k included and board power sampled during the timed loop costs 1.22 ms and 277 mJ per token at batch 1, and 21 μ s and 4.7 mJ per token at batch 64. Achieved bandwidth was 1.6–1.8 TB/s, about half of the part’s specification, so the $K \approx 204\text{K}$ crossing of Sec. III is closer to 107K in practice. Compression also helps the GPU, and the comparison should say so: the same 384-bit codes the CAM holds, searched on the GPU with a population-count kernel (FAISS), cost 28 μ s and 2.1 mJ per token at batch 1 (0.45 mJ above idle) and 0.5 μ s and 0.04 mJ at batch 64, flat in K from 10^3 to 131K rows – the scan itself is under 30 μ s, and the GPU sits at about 3% of its per-token budget at 10^3 tokens/s. What the array offers over that baseline is the search energy itself (0.03–0.3 μ J against 40 μ J–2 mJ

⁹What the estimate omits, on each side: the GPU figure excludes the distance arithmetic itself and any cache reuse across batched requests (Sec. VII-F); the CAM figure takes array-level published values, which include matchline and sense-amp energy but not the query projection ($\sim 10^6$ multiply–accumulates per token, negligible at this scale) or the mismatch-count reduction across subarrays. These are first-order literature values, not our measurements.

TABLE VI: The cost model: measured rows, first-order estimates, and open entries. EST marks first-order values from literature figures (HBM at 3.97 pJ/bit [43]; CAM cells at 0.1–1 fJ/search [19], [20], [44]), at $K = 1.3 \times 10^5$, 2,048-bit rows, per token per layer. MEAS marks measurements on one H100 PCIe at batch 1 / batch 64; TBD entries require the measurements of Sec. VII-F.

Metric	CAM	GPU	Assumptions
Energy / search	0.03–0.3 μ J EST	277 / 4.7 mJ MEAS (~ 67 mJ EST)	cell figure spans published range; EST is DRAM access only
Latency / search	0.3–4.4 ns/arr. EST	1.22 ms / 21 μ s MEAS (0.63 ms EST)	reduction net. open; EST is the batch-1 streaming floor
GPU on 384-b codes	n/a	2.1 / 0.04 mJ; 28 / 0.5 μ s MEAS	FAISS popcount, flat in K from 10^3 to 131K
Area / 10^3 rows	TBD	n/a	cell type (Table I), node
Coverage / mm ² , / W	TBD	TBD	behaviors $\times$ layers, at fixed detection rate
Traffic / token	dbL_{mon}	$K dbL_{\text{mon}}$	measured here; Eqs. (2), (3)
Bank capacity		Fig. 6	measured here; code widths from Sec. V

per token per layer) and freedom from contention with live inference, rather than bandwidth relief alone. Table VI carries these entries, the CAM rows marked as estimates, alongside the rows that remain open.

F. The measurements that would replace the estimates

Each estimate above maps to a specific, short measurement, and the tooling for the array side already exists in-house. (1) **Array-level characterization with Eva-CAM:** Eva-CAM is a circuit/architecture-level evaluation tool for TCAM, MCAM and ACAM arrays across NVM technologies, validated against fabricated chips to within 6–20% [44]. Running it at a stated node for exactly the configurations this paper measures – 384- and 2,048-bit rows, banks of 10^3 – 1.3×10^5 rows, BCAM, TCAM and 3-bit MCAM cells – would replace the EST entries of Table VI with modeled energy, latency and area, including the subarray split and the reduction network the footnote above leaves open. This is days of work with an existing tool. (2) **A tuned GPU baseline:** the H100 measurements of Sec. VII-E (fp16 scan and FAISS popcount [33], batch 1 and 64, measured power) are now in Table VI; the Jetson-class part remains to be measured, so the amortization a datacenter part enjoys – and an edge part does not – can appear in the table rather than in a caveat. Recall for any approximate index should additionally be measured under queries chosen *against* the index rather than drawn at random, since that is the regime a monitor operates in [45]. (3) **The fidelity check the encoding comparison still needs:** matched cell count (Sec. V-B) and a common frozen projection on the deception grid (Sec. V-C) are done; a common projection across the Hamming and Euclidean encodings on the readout workload remains, and

it is what settles whether the cell-type flip of Sec. VII-D is a device result or a projection artifact. It is a cluster run on data already in hand.

VIII. LIMITATIONS AND CONCLUSIONS

A. Limitations

Several qualifications apply to everything above.

What we have not built: Nothing here competes with a deployed classifier, and no end-to-end jailbreak-blocking experiment was run. The measured claims concern the fidelity and cost of stored-signature matching, which is one component of the always-on stage described in Sec. I.

The label: The experiments define deception as an output that contradicts the known correct output. Deception in the stronger sense – a model representing a claim as false while asserting it – is harder to label, and the available datasets control for it imperfectly.

Granularity: The grid results classify one activation per response rather than per token, while the deployment target the hardware serves is per-token and always-on. The watchlist reproduction of Sec. VI-B is per-token, which is part of why it is reported here.

Layer choice and model coverage: Layer 33 is a tuned hyperparameter of one model family at one scale, and re-tuning per model is assumed. All grid results are one model. Re-tuning is also what makes the bank a rewritten structure, which is why endurance appears in Table V.

Hardware numbers: The comparison in Sec. VII is capacity and bandwidth arithmetic on vendor figures, with no contention term for a monitor co-resident with live inference. The energy and latency entries of Table VI are first-order estimates from literature values, not our measurements; Sec. VII-F lists what would replace them.

Three metrics: AUROC values in Sec. V are cross-domain on the ten-domain grid; the first pipeline’s numbers in Sec. IV-B are in-distribution on a four-domain grid under a different protocol; and the readout numbers in Sec. VI are F1 and top-1 agreement against a full-precision reference rather than against labels. None of the three is comparable with the others cell by cell, and the paper does not place them side by side.

Digitized values: the readout fidelity in Fig. 5 was digitized from result charts rather than read from result files, and the library-size sweep carries three stated implementation limitations (Sec. VI-D). Both should be regenerated from source before submission.

B. Conclusions

Always-on behavioral monitoring of a language model is a library-search problem, and the variable that decides both its coverage and its cost is K , the number of stored signatures compared against each activation. At $K = 1$, which is what is deployed today, the operation is nearly free and no special hardware is warranted. Published work gives reasons to expect K to grow – non-transfer across behaviors, steering wanting

a direction per behavior per domain per layer, feature dictionaries, and whole-vocabulary readout, which fixes $K \approx 10^5$ per layer by construction – and also gives reasons to doubt it, which is why the question is worth measuring rather than asserting.

One methodological result comes before the fidelity numbers. Accuracy alone does not establish that a memory-based detector is using its memory: across five variants of an earlier pipeline, 66–100% of the accuracy of the two best-scoring configurations survived randomizing every stored row, so the trained front end was doing most of the classifying and the array was close to decoration. Reporting front-end share alongside AUROC separates the two, and it is what moved this work onto frozen projections.

What a CAM-resident monitor gives up can then be measured, and we measure it on the two workloads at opposite ends of the range. For deception detection at 10^2 – 10^3 labeled rows, a 384-bit code inside a plain array holds in-domain AUROC within 0.009 of full-precision cosine similarity on the raw 8,192 numbers, at 0.920 (sign-RP ternary) and 0.919 (thermometer) against 0.928; cross-domain, the same codes give up 0.065 and 0.127. At a matched 384 bits the ternary code is ahead of the thermometer code by 0.061 on average, but at matched cell count (thermometer at 768 b) the two are within 0.009 on the cross-domain mean, and a frozen projection on a multi-bit Euclidean matchline buys no accuracy over Hamming distance, so on this grid the cheaper cell is sufficient. A two-stage shortlist recovers most of the cross-domain gap in aggregate, but its fixed k does not travel across bank sizes, which argues for a threshold criterion over a top- k one in any array that has to serve banks of different sizes.

For whole-vocabulary readout at 10^5 rows per layer by construction, where the exact readout is its own ground truth and no labels are needed, a 3-bit Euclidean code at $\sim 10^3$ storage bits per row – a $128\times$ compression of an fp16 row – reproduces 0.96 of the exact top-1 selections and 0.81 of the per-row firing decisions. Reporting the closest row survives compression better than reporting which rows cross a threshold, by 0.15–0.2 at every code width, which is a matchline-mode result rather than a cell-type one. Swept over library size, the bank stays alive to 10^4 rows while per-row fidelity falls from 0.755 to 0.43 – and falls faster in full precision than at 3 bits, so what gets harder with K is the discrimination rather than the arithmetic.

On the cost side, monitoring traffic on a GPU crosses an H100’s entire HBM bandwidth at $K \approx 204K$ and a Jetson-class part’s at $K \approx 17K$, while the same library at the measured code widths is 6–33 MB per layer and fits on a die. A first-order estimate from literature figures puts the energy gap of the search itself at five to six orders of magnitude – roughly 67 mJ per token per layer to stream the whole-vocabulary bank from HBM against 0.03–0.3 μ J to compare it in place – of which a factor of 64 is the measured compression and the rest is the substrate (Sec. VII-E). A library that is inexpensive to hold and expensive to move is the condition under which searching it in place is worth the silicon. The remaining work

is short and specific: Eva-CAM characterization at a stated node to turn the array estimates into measurements, now that the GPU side is measured (Sec. VII-F), and – upstream of any hardware – establishing the minimum K that adequate behavioral coverage actually requires.

REFERENCES

- [1] METR, “Common elements of frontier safety policies,” METR, 2026. [Online]. Available: <https://metr.org/common-elements>
- [2] Anthropic, “Constitutional classifiers++: Efficient production-grade defenses against universal jailbreaks,” *arXiv preprint arXiv:2601.04603*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.04603>
- [3] —, “Next-generation constitutional classifiers,” Anthropic research, 2026. [Online]. Available: <https://www.anthropic.com/research/next-generation-constitutional-classifiers>
- [4] Google DeepMind, “Building production-ready probes for gemini,” *arXiv preprint arXiv:2601.11516*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.11516>
- [5] UK AI Security Institute, “Our evaluation of claude mythos preview’s cyber capabilities,” AISI, 2026. [Online]. Available: <https://www.aisi.gov.uk/blog/our-evaluation-of-claude-mythos-previews-cyber-capabilities>
- [6] —, “Incident report: Unsanced agent behavior during cyber testing,” AISI, 2026, covering 25–28 July 2026. [Online]. Available: <https://www.aisi.gov.uk/blog/incident-report-unsanced-agent-behaviour-during-cyber-testing>
- [7] J. Clark, “Import AI 471: Why Hugging Face worries me; space mining; Five Eyes on AI,” Import AI, 2026, august 31, 2026. [Online]. Available: <https://importai.substack.com/p/import-ai-471-why-hugging-face-worries>
- [8] A. Cotra, “The Hugging Face attack surprised me,” Planned Obsolescence, 2026. [Online]. Available: <https://www.planned-obsolence.org/p/the-hugging-face-attack-surprised>
- [9] S. Marks and M. Tegmark, “The geometry of truth: Emergent linear structure in large language model representations of true/false datasets,” *arXiv preprint arXiv:2310.06824*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.06824>
- [10] C. Burns, H. Ye, D. Klein, and J. Steinhart, “Discovering latent knowledge in language models without supervision,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [11] A. Zou *et al.*, “Representation engineering: A top-down approach to ai transparency,” *arXiv preprint arXiv:2310.01405*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.01405>
- [12] Z. J. Ying, S. Ravfogel, N. Krieger, and P. Hase, “The truthfulness spectrum hypothesis,” *arXiv preprint arXiv:2602.20273*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.20273>
- [13] T.-F. Berger, “Probing the limits of the lie detector approach to LLM deception,” *arXiv preprint arXiv:2603.10003*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.10003>
- [14] Schmidt Sciences, “AI interpretability program: Detecting deceptive reasoning in LLMs,” Schmidt Sciences, 2026. [Online]. Available: <https://www.schmidtsciences.org/ai-interpretability/>
- [15] Anthropic, “A global workspace in language models,” Anthropic research, 2026, july 6, 2026. [Online]. Available: <https://www.anthropic.com/research/global-workspace>
- [16] A. Templeton *et al.*, “Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet,” *Transformer Circuits Thread*, 2024. [Online]. Available: <https://transformer-circuits.pub/2024/scaling-monosemanticity/>
- [17] L. Gao *et al.*, “Scaling and evaluating sparse autoencoders,” *arXiv preprint arXiv:2406.04093*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.04093>
- [18] T. Lieberum *et al.*, “Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2,” *arXiv preprint arXiv:2408.05147*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.05147>
- [19] K. Pagiamtzis and A. Sheikholeslami, “Content-addressable memory (CAM) circuits and architectures: A tutorial and survey,” *IEEE Journal of Solid-State Circuits*, vol. 41, no. 3, pp. 712–727, 2006.
- [20] K. Ni *et al.*, “Ferroelectric ternary content-addressable memory for one-shot learning,” *Nature Electronics*, vol. 2, pp. 521–529, 2019.

- [21] A. Kazemi *et al.*, “In-memory nearest neighbor search with FeFET multi-bit content-addressable memories,” in *Design, Automation and Test in Europe (DATE)*, 2021. [Online]. Available: <https://arxiv.org/abs/2011.07095>
- [22] C. Li *et al.*, “Analog content-addressable memories with memristors,” *Nature Communications*, vol. 11, no. 1, p. 1638, 2020.
- [23] A. Kazemi, F. Müller, M. M. Sharifi, H. Errahmouni, G. Gerlach, T. Kämpfe, M. Imani, X. S. Hu, and M. Niemier, “Achieving software-equivalent accuracy for hyperdimensional computing with ferroelectric-based in-memory computing,” *Scientific Reports*, vol. 12, p. 19201, 2022.
- [24] S. Dutta *et al.*, “Monolithic 3D integration of high endurance multi-bit ferroelectric FET for accelerating compute-in-memory,” in *IEEE International Electron Devices Meeting (IEDM)*, 2020.
- [25] K.-H. Kim *et al.*, “Scalable CMOS back-end-of-line-compatible AlScN/two-dimensional channel ferroelectric field-effect transistors,” *Nature Nanotechnology*, vol. 18, 2023.
- [26] GSI Technology, “Gemini apu: Enabling high-performance billion-scale similarity search,” GSI Technology, 2025. [Online]. Available: <https://gsitechnology.com/gemini-apu-enabling-high-performance-billion-scale-similarity-search/>
- [27] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024, 671B-parameter MoE; $d = 7,168$, 61 layers. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [28] M. Sharma *et al.*, “Constitutional classifiers: Defending against universal jailbreaks,” *arXiv preprint arXiv:2501.18837*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.18837>
- [29] Anthropic Alignment Science, “Cheap monitors,” Anthropic alignment blog, 2025. [Online]. Available: <https://alignment.anthropic.com/2025/cheap-monitors/>
- [30] P. Indyk and R. Motwani, “Approximate nearest neighbors: Towards removing the curse of dimensionality,” in *ACM Symposium on Theory of Computing (STOC)*, 1998, pp. 604–613.
- [31] M. S. Charikar, “Similarity estimation techniques from rounding algorithms,” in *ACM Symposium on Theory of Computing (STOC)*, 2002, pp. 380–388.
- [32] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [33] Meta Engineering and NVIDIA, “Accelerating gpu indexes in faiss with nvidia cuvs,” Meta Engineering blog, 2025. [Online]. Available: <https://engineering.fb.com/2025/05/08/data-infrastructure/accelerating-gpu-indexes-in-faiss-with-nvidia-cuvs/>
- [34] NVIDIA, “NVIDIA H100 tensor core GPU datasheet,” NVIDIA, 2023, 3.35 TB/s HBM3, 50 MB L2. [Online]. Available: <https://resources.nvidia.com/en-us-tensor-core>
- [35] Lyceum Technology, “LLM inference tokens per second: 2026 benchmarks,” Online benchmark roundup, 2026. [Online]. Available: <https://lyceum.technology/magazine/llm-inference-tokens-per-second-comparison-2026/>
- [36] W. Jiang, J. Clemons, K. Sankaralingam, and C. Kozyrakis, “How fast can I run my VLA? demystifying VLA inference performance with VLA-perf,” *arXiv preprint arXiv:2602.18397*, 2026, nVIDIA Research; π_0 at 19 Hz on Jetson Thor, 50-action chunks. [Online]. Available: <https://arxiv.org/abs/2602.18397>
- [37] S. Ravfogel, Y. Elazar, H. Gonen, M. Twiton, and Y. Goldberg, “Null it out: Guarding protected attributes by iterative nullspace projection,” in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [38] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, “Learned step size quantization,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [39] Apple, “Apple intelligence foundation language models,” *arXiv preprint arXiv:2507.13575*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.13575>
- [40] “A dual-mode 1FeFET-1C embedded memory: Characterization, eDRAM benchmarking, and a ternary CAM proof of concept,” 2026, manuscript in preparation.
- [41] Llama Team, AI @ Meta, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [42] NVIDIA, “Jetson AGX Thor developer kit,” NVIDIA product documentation, 2025, 273 GB/s LPDDR5X, 128 GB, 130 W. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-thor>
- [43] M. O’Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, “Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems,” in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, hBM2 access energy ≈ 3.97 pJ/bit.
- [44] L. Liu, M. M. Sharifi, R. Rajaei, A. Kazemi, K. Ni, X. Yin, M. Niemier, and X. S. Hu, “Eva-CAM: A circuit/architecture-level evaluation tool for general content addressable memories,” in *Design, Automation and Test in Europe (DATE)*, 2022.
- [45] A. Andoni, T. Haris, E. Kelman, and K. Onak, “Efficient algorithms for adversarially robust approximate nearest neighbor search,” *arXiv preprint arXiv:2601.00272*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.00272>